

Contents

vLLM Serving Performance Analysis: Phase-Level Measurement of Prefill and Decode	1
Abstract	1
1. Introduction	1
2. Background & Related Work	1
3. Methodology	2
4. Results	8
5. Bottleneck Analysis	11
6. Threats to Validity	12
7. Conclusion	12
References	12
Appendix	12

vLLM Serving Performance Analysis: Phase-Level Measurement of Prefill and Decode

Issei Hasegawa — Allegheny College Repository: <https://github.com/IsseiHasegawa/vLLM-experiment>

= must fix before submission = add only if the pp=2 session is executed; delete the marker otherwise
WRITING ORDER: 3 -> 4 -> 5 -> 6 -> 1 -> Abstract -> 2 ->

Abstract

1. Introduction

RQ1. As the request arrival rate increases, how do per-phase latency and throughput change, and where and in what form does the capacity limit appear?

RQ2. How do workload characteristics (dataset, input/output ratio) and model size change capacity and phase composition?

RQ3. How large is the gain from additional GPUs and parallelism strategy, and to which phase is that gain attributable?

RQ4. Which resource determines the limits observed above?

2. Background & Related Work

2.1 Prefill and decode

2.2 Related work

3. Methodology

3.1 Instrumentation

For this measurement, we forked vLLM from GitHub (vllm-project/vllm, base commit 702f4814), made changes to three files totaling approximately 190 lines on the instrumentation branch, and then built it using an editable install from the source. Inference paths have never been changed except for instrumentation. The main purpose of this research is not optimization but measurement. All of these changes are intended to make the internal time readable from the outside. The fork, the instrumentation commit (019e5d1), and the buffered-flush commit (d4e0675) are available at github.com/IsseiHasegawa/vllm.

The design of Instrumentation is not to build a new timer, but to write out the value that vLLM V1 has already calculated. The V1 metrics layer retains the times for queuing, prefilling, and decoding for each request at the time `FinishedRequestStats` is constructed. By adopting an approach that outputs this data directly to JSONL without recalculating it, we minimized the risk of introducing bugs through custom code and, at the same time, enabled cross-validation with the Prometheus histograms published by vLLM itself.

The core of this design lies in dividing the instrumentation into two axes. In vLLM V1, chunked prefill is always enabled, and a single engine step contains a mix of requests undergoing prefill and requests undergoing decoding. In other words, phase refers to a request attribute, while step refers to a batch containing a mix of both phases. Since the two have different levels of granularity, relying on only one of them would not allow time and resource usage to be separated by phase. Therefore, we adopted a three-layer architecture that independently logs the request and step axes and samples resources from outside the process at 1 Hz (Figure 1).

Table 1: Schema of the three instrumentation layers.

Layer	Output	Fields
Request axis	<code>requests.jsonl</code>	<code>queued_s</code> , <code>prefill_s</code> , <code>decode_s</code> , <code>inference_s</code> , <code>e2e_s</code> , <code>n_prompt</code> , <code>n_gen</code> , <code>n_cached</code>
Step axis	<code>steps.jsonl</code>	<code>sched_s</code> , <code>exec_s</code> , <code>n_ctx_reqs</code> , <code>n_ctx_toks</code> , <code>n_gen_reqs</code> , <code>n_gen_toks</code> , <code>n_running</code> , <code>n_waiting</code> , <code>kv_usage</code>
Resource	external logger	SM utilisation, memory-controller utilisation, VRAM, power (pynvml); per-process CPU utilisation for the server and the benchmark client (psutil)

The definitions of each time period were adopted directly from the vLLM’s internal definitions. `queued_s` covers the period from the `QUEUED` event to the first `SCHEDULED` event; `prefill_s` covers the period from the first `SCHEDULED` event to the first token (including chunk segmentation and any waiting time in between); `decode_s` covers the period from the first token to the last token; and `e2e_s` covers the period from arrival at the front end to completion. Since the frontend and engine core are separate processes, the output destinations are also separated by process.

The logger is enabled only when the environment variable `VLLM_PHASE_LOG_DIR` is set; if it is not set, the function immediately returns at the beginning of the hook. Because instrumentation can be disabled without changing the binary, this enables the C1 control experiment described later (§3.5). Writing is performed via a buffer, and a background flush is scheduled every second to ensure that the last record is not lost in the event of a server crash.

The correctness of the instrumentation was verified through actual measurements. First, we verified the identity `prefill_s + decode_s = inference_s`—which should hold given the configuration—for all records, and then cross-checked the results against vLLM’s own Prometheus histograms. In the initial validation on a 0.5B model, the queueing, prefill and decode means agreed exactly (1.07 ms, 25.43 ms and 714.32 ms against identical Prometheus values over 60 requests), but the step-level token accounting fell short of the request-level total, consistent with buffered records lost at shutdown. This motivated the background flush described above. After that change, the same battery on session B reproduced the Prometheus agreement (72.04 ms, 238.24 ms, 2135.56 ms) and the token accounting matched exactly: 6,400 prefill tokens and 3,150 decode tokens on both axes. Across the three measurement sessions the instrumentation wrote 46,229 request records and 756,653 step records. The per-session request counts (20,440, 7,700 and 18,089) each matched the expected total exactly, confirming that no records were lost at shutdown.

Additionally, we implemented two safeguards to ensure that the attribution of timing measurements remained unambiguous. First, there are two execution paths for vLLM steps: `step` and `step_with_batch_queue`; in the latter, execution overlaps with the next step. In this measurement, we explicitly disabled async scheduling and ensured that all runs followed the former path by having the patch log which path was selected at startup once. Second, we did not use the `--disable-log-stats` option. This is because phase timestamps are carried as `EngineCoreEvents`, and disabling statistics would cause the measurement targets themselves to disappear.

3.2 Workloads and datasets

For this measurement, we selected two datasets with contrasting characteristics. ShareGPT is a public dataset comprising actual interactions between humans and dialogue models, and the prompt lengths vary widely. `random` is a synthetic workload generated by the vLLM benchmark harness, where the input and output lengths can be fixed. We chose these two because, since the amount of work required for prefill depends on the prompt length, the distribution of prompt lengths itself becomes an independent variable. ShareGPT is realistic but uncontrollable, while `random` is unrealistic but fully controllable. The former provides external validity, while the latter provides internal validity.

ShareGPT used `ShareGPT_V3_unfiltered_cleaned_split.json` (672,837,942 bytes; first 16 digits of SHA-256 hash: 35f0e213ce091ed9). Distribution statistics were calculated using the Qwen2.5-7B-Instruct tokenizer, with 5,000 samples extracted using seed 42. The `random` workload used a default of 256 input and 128 output tokens, with `range_ratio` set to 0 to eliminate length variability. In groups I1 and I2, which isolate the effects of the input-to-output ratio, this fixed length was changed to 512/128 and 128/512, respectively.

The nominal and realised distributions differ substantially. The harness sampler admits a conversation only if its prompt and output are each at least 4 tokens, the prompt is at most 1,024 tokens, and prompt and output together are at most 2,048 (`is_valid_sequence` in `vllm/benchmarks/datasets/datasets.py`). The source file reaches 66,076 tokens, but the realised maximum is 1,010 — a consequence of this filter, not of the data. The summaries of both are shown side by side.

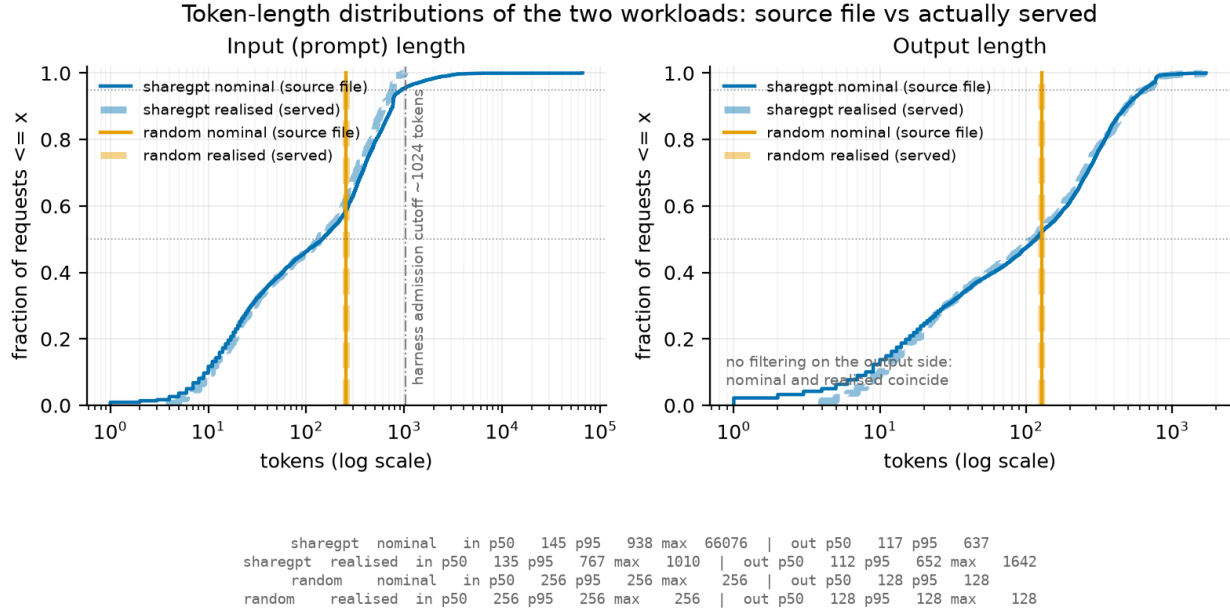


Figure 2. Cumulative distribution of prompt lengths for ShareGPT and random (log x-axis). Solid lines are realised values taken from the phase logs; dashed lines are nominal values from the source file. The divergence of the two ShareGPT curves at 1,024 tokens is caused by the benchmark harness sampler, which does not admit longer prompts. The random workload is fixed-length and therefore steps.

Table 2: Prompt-length distributions, nominal versus realised.

Dataset	Source	Input p50	p95	max
ShareGPT	nominal (source file, n=5,000)	145	938	66,076
ShareGPT	realised (S1 phase logs, n=4,800)	136	767	1,010
random (S2 default)	nominal = realised (fixed)	256	256	256

Showing only the nominal distribution would result in an overestimation of the actual processed tail by a factor of 65. Therefore, in Figure 2, both distributions are overlaid, and the upper limit of 1,024 tokens is explicitly indicated. The claim of a realistic conversational workload in this report is limited to the range within this upper bound.

Caution is also required regarding output length. ShareGPT’s realised output lengths have a p95/p50 ratio of 5.8 with a maximum of 1,642 tokens (S1, n = 4,800), i.e. a heavy tail. Under offline conditions (rate = inf), a run ends when its longest request completes, so S1’s achieved throughput at rate = inf varies by 41 % across seeds (3.23 / 4.57 / 3.56 req/s). For this reason, we do not derive ShareGPT’s capacity from rate = inf; instead, dataset comparisons are made at matched rates or from the closed-loop group C2.

We specified `-ignore-eos` for all runs, which pins the generation length to the requested value. This ensures that the output length is no longer influenced by the model’s decision to stop, thereby standardizing the workload across conditions. With prefix caching disabled (§3.3), no KV state is

shared between requests even when prompts repeat across repetitions, so the repetitions remain independent. We verified `n_cached = 0` across all 46,229 request records.

3.3 Experiment matrix

A total of 208 runs were executed; 207 completed successfully and form the dataset used throughout this report. One run (G1 at rate 1, repetition 1) returned 199 of the 200 requested completions and was recorded as a failure. It was not re-run, so that single point rests on two repetitions rather than three. The matrix was generated using `scripts/make_matrix.py`. We chose to generate the matrix from code rather than using a hand-written CSV file so that additions or modifications to the conditions would be recorded as changes in the history, ensuring reproducibility. Each condition was repeated three times, with the seed changed to 1, 2, and 3 for each iteration (§3.4).

The table below shows the main groups and the variables each group controls.

Table 3: Experiment groups. 208 runs executed, 207 analysed.

Group	Model	Dataset	GPUs	Varied	Runs analysed
S1	7B	ShareGPT	1	arrival rate	24
S2	7B	random	1	arrival rate	24
S2b	7B	random	1	arrival rate, extended	15
P0	0.5B	ShareGPT	1	offline capacity probe	1
S3	0.5B	ShareGPT	1	arrival rate	27
I1 / I2	7B	random	1	input/output ratio	3 / 3
G1 / G2 / G4	7B	ShareGPT	1 / 2 / 4	GPU count, tp	23 / 24 / 24
C2 / C2x	7B	ShareGPT	1	concurrency, closed loop	21 / 3
C1off	7B	ShareGPT	1	instrumentation on/off	3
A1a–A1d	7B	ShareGPT	1	anchor across sessions	12

The arrival rate grid was varied for each model. For 7B, it was $\{1, 2, 3, 4, 5, 6, 8, \infty\}$ req/s, and for 0.5B, it was $\{1, 2, 4, 8, 12, 16, 24, 32, \infty\}$ req/s. There is approximately a sixfold difference in capacity between the two models, and the 0.5B saturation point cannot be captured within the same grid. This conclusion is based on measurements, not speculation. Prior to the S3 sweep, a single probe run, P0, was executed under offline conditions ($\text{rate} = \infty$) to confirm that the 0.5B capacity falls within the expanded grid. Similarly, the original grid for S2 was too small for the capacity of the random workload and did not reach the saturation knee. Therefore, in S2b, we added $\{5, 10, 12, 16, 20\}$ req/s and connected them to the S2 series with rate 5 as a duplicate point.

Each open-loop run submitted 200 requests. Only C2 in the closed-loop experiments varied the number of prompts according to the concurrency level: 60 prompts at concurrency 1 and 2, 120 at 4 and 8, and 200 at 16 and above. At concurrency 1 requests are served one at a time, so even the reduced count of 60 still took 20.5 minutes per run; keeping it at 200 would not have been practical. This represents a trade-off between accuracy per point and total elapsed time.

The following settings are common to all runs to ensure comparability between conditions. The `-num-warmups 30` option ensures that measurements are taken only after the server has reached a steady state. `-temperature 0` disables sampling based on the server’s default generation settings, making the runs reproducible. `-ignore-eos` pins the generation length to the requested value (§3.2). `-no-enable-prefix-caching` preserves independence between iterations (§3.2). The recorded percentiles were 50, 95, and 99.

Comparisons of the number of GPUs were performed within a single instance. G1, G2, and G4 were all measured on the same 4-GPU machine; the only difference between the series was the parallelization settings. If measurements with different `tp` values were taken on separate instances, it would be impossible to separate the effects of host differences from those of parallelization. For the same reason, although `tp=1` was also measured in Session A, it was remeasured as G1.

The measurements are divided into three sessions. Session A ($1 \times A40$, 91 runs) covers single-GPU sweeping and the I/O ratio; Session B ($1 \times A40$, 39 runs) covers closed-loop and S2b; and Session C ($4 \times A40$, 78 runs) covers the number of GPUs and parallelization. The validity of cross-session comparisons is discussed in §3.5. The hardware used was the NVIDIA A40 48GB, and the driver was standardized to the 580 series (CUDA 13.0).

The pipeline-parallel group P1 (`pp=2`, 24 runs) is defined in the matrix but is not included in the measurement scope of this report.

3.4 Metrics

This report uses four request-level latency metrics and several throughput metrics. The definitions are consistent across the benchmark harness and instrumentation logs.

TTFT (time to first token) is the time from when a request arrives at the front end until the first token is returned, reflecting the weight of the prefill phase. TPOT (time per output token) is the average time per token after the first token, while ITL (inter-token latency) is the distribution of token intervals itself; both reflect the speed of the decode phase. While TPOT is averaged over a single request, ITL retains individual intervals; therefore, intermittent stalls caused by scheduling appear at the tails of the ITL distribution, whereas they are smoothed out and become invisible in TPOT. E2EL (end-to-end latency) is the total time from arrival to completion, including both phases and queues.

As a general rule, latency distributions are reported using p50 and p95 rather than means. As shown in §3.2, ShareGPT’s output length has a p95/p50 ratio of 5.8, indicating a heavy tail, and the average is dominated by a small number of long requests. By reporting the middle and the tail separately, we can distinguish whether an increase in load affects typical requests or only the tail. The exception is the TTFT decomposition in §4.2. There, TTFT is split into a queueing component and a prefill component; however, since percentiles are not additive, the components would not sum to the total. For this figure only, we use means and state this explicitly in the figure.

Since there are two possible definitions of achieved throughput, this report presents both.

The first definition is the `request_throughput` reported by the harness, i.e., the number of completions divided by the measurement time. Since the measurement time includes the period until the last request is completed, the drain after arrivals have stopped is added to the denominator. In S1, this drain lasted approximately 12 seconds at rate 1 and 32 seconds at rate 8. As a result, the achieved rates appear lower at all points, including those where the server is keeping up comfortably.

The measured value for the nominal Rate 1 was 0.95 req/s.

The second definition is the number of completions by the time of the last arrival divided by the arrival span. While this eliminates the impact of drain, it introduces a reverse bias—requests that were still being processed when arrivals ceased are not counted—and the magnitude of this bias increases with load.

Neither definition is unbiased. Plotting both is a deliberate choice to demonstrate that achieved throughput is a metric sensitive to the definition used. In particular, in this system, since the vLLM accepts all incoming requests into the currently processing batch, congestion manifests as an increase in batch size rather than queue length. Therefore, achieved throughput is a weak indicator of saturation. Claims regarding saturation rely on latency (Figure X) and the closed-loop curve (Figure Y).

For token-level throughput, we report both `output_throughput`, which counts only output tokens, and `total_token_throughput`, which sums both inputs and outputs. Since the former represents productivity on the decode side and the latter represents the total workload including prefill, the difference between the two serves as the result when comparing conditions with different input-to-output ratios (I1 and I2).

Each condition was repeated three times, and the error bars in the figure represent the standard deviation of the three data points. Since the seed was changed to 1, 2, and 3 for each iteration and the same set of seeds was reused across all conditions, comparisons between conditions are directly comparable. Due to this design, the error bars include not only system noise but also variations in prompt sampling and arrival jitter.

3.5 Measurement validity

The validity of this measurement depends on two assumptions: that the instrumentation itself does not alter the subject of measurement, and that the results across the three sessions are comparable. Both assumptions were verified through controls incorporated into the design.

Instrumentation overhead (C1). The same conditions were measured with the instrumentation enabled (A1a) and disabled (C1off). Since the logger switches based on an environmental variables (§3.1), both arms run the same binary. Both were executed in close temporal proximity within Session A and used the same set of seeds.

A paired t-test detected a difference of +2.97% in TTFT p50 ($p = 0.010$, $n = 3$ seed pairs). When the same comparison is made without pairing, the difference is nearly the same at +3.06%, but the p-value is 0.687, so the same difference is undetectable. This is because an independent test has virtually no statistical power with $n = 3$; the design that pairs seeds (§3.4) is effective here.

More important than the magnitude of the difference is its direction. All seven metrics moved in the same direction—enabling instrumentation results in slower response times and lower throughput. While many individual metrics fall within the range of practical equivalence, the fact that they all point in the same direction is evidence of a real effect. Therefore, we do not claim that the effect of instrumentation is negligible. The latency values in this report should be interpreted as upper bounds, and the throughput values as lower bounds, with a margin of 2.97%. Since all measurements were taken with instrumentation enabled, this bias applies uniformly across all conditions and does not affect comparisons between conditions.

Comparability across sessions (A1 anchor). The same conditions—7B / ShareGPT / single GPU /

rate 5—were repeated at four different time points. A1a and A1b correspond to the beginning and end of Session A (same instance, spanning 3 h 13 min from the start of A1a to the end of A1b); A1c corresponds to Session B; and A1d corresponds to Session C. The three sessions were run on different instances spanning two geographic regions.

Table 4: Anchor runs across sessions (7B, ShareGPT, 1 GPU, rate 5, $n = 3$ each).

Anchor	Session	Achieved throughput (req/s)	vs A1a
A1a	A	3.195	—
A1b	A	3.205	+0.31 %
A1c	B	3.174	−0.66 %
A1d	C	3.187	−0.25 %

The overall spread across the four points is 0.97%. Given that the difference between A1a and A1b—which bracket a three-hour block on the same instance—is 0.31%, the differences across sessions and regions are of a magnitude comparable to the drift within a single instance. The performance differences compared in this report (such as the +32.5% gain from tensor parallelism and capacity differences due to the dataset) are more than thirty times larger than this drift, so comparisons across sessions are acceptable.

Note that the comparison of the number of GPUs does not depend on this tolerance range. Measurements for G1, G2, and G4 were all taken within a single instance (§3.3). The anchor serves to discuss the sweep in Session A, the closed-loop in Session B, and the results of Session C within the same context.

4. Results

4.1 Effect of arrival rate

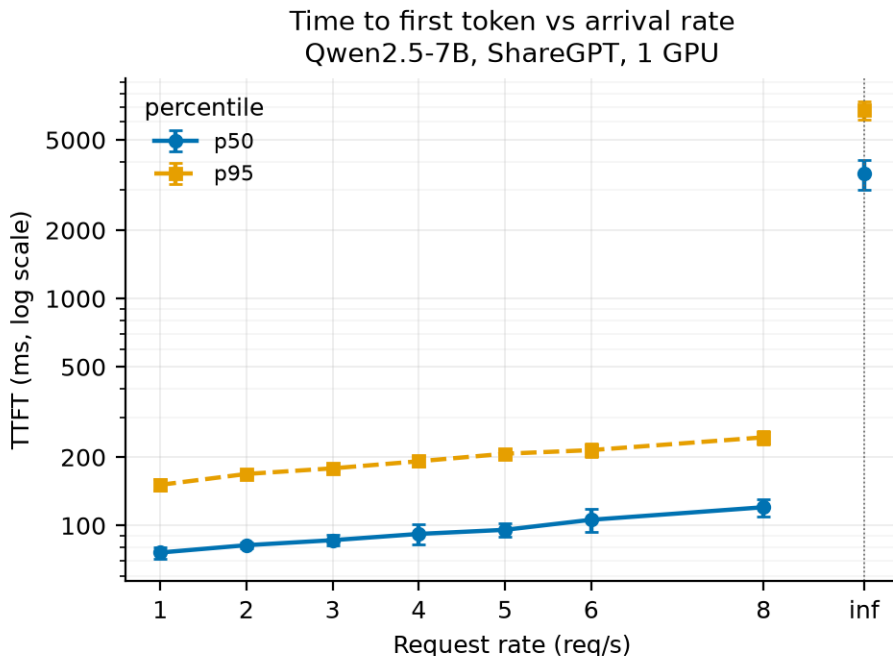


Figure 3.

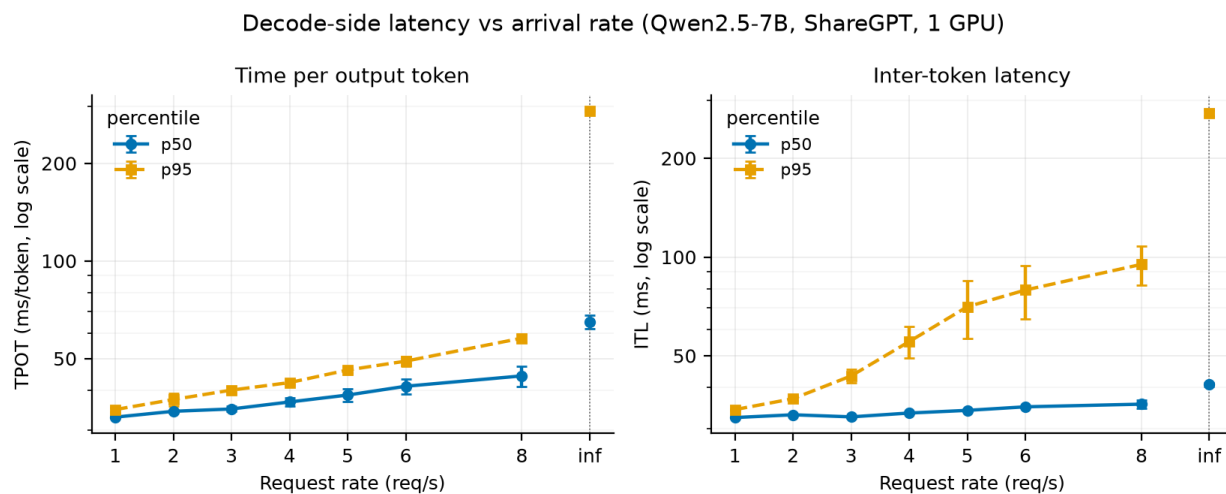


Figure 4.

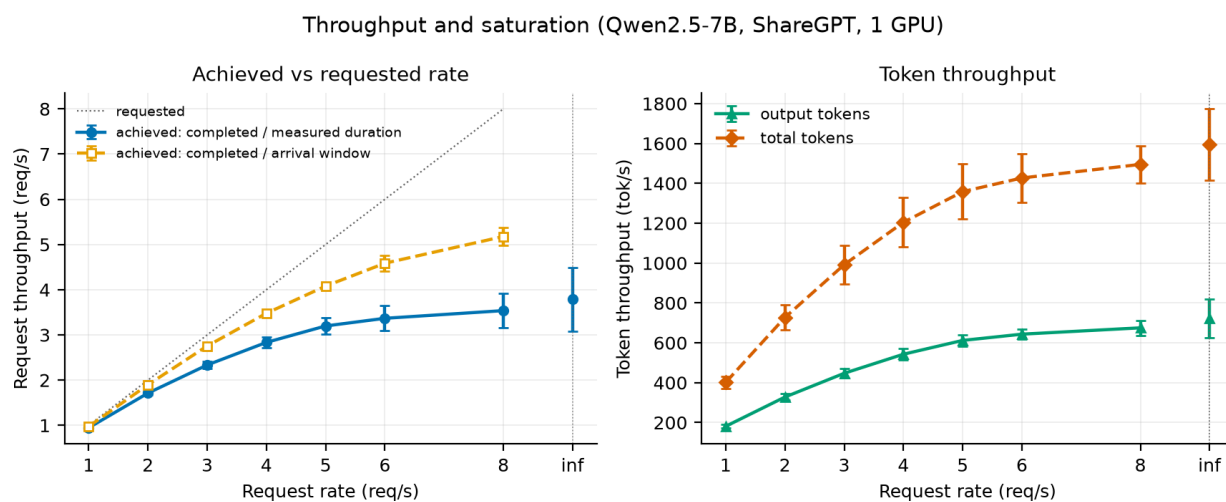


Figure 5.

4.2 Phase-level behaviour

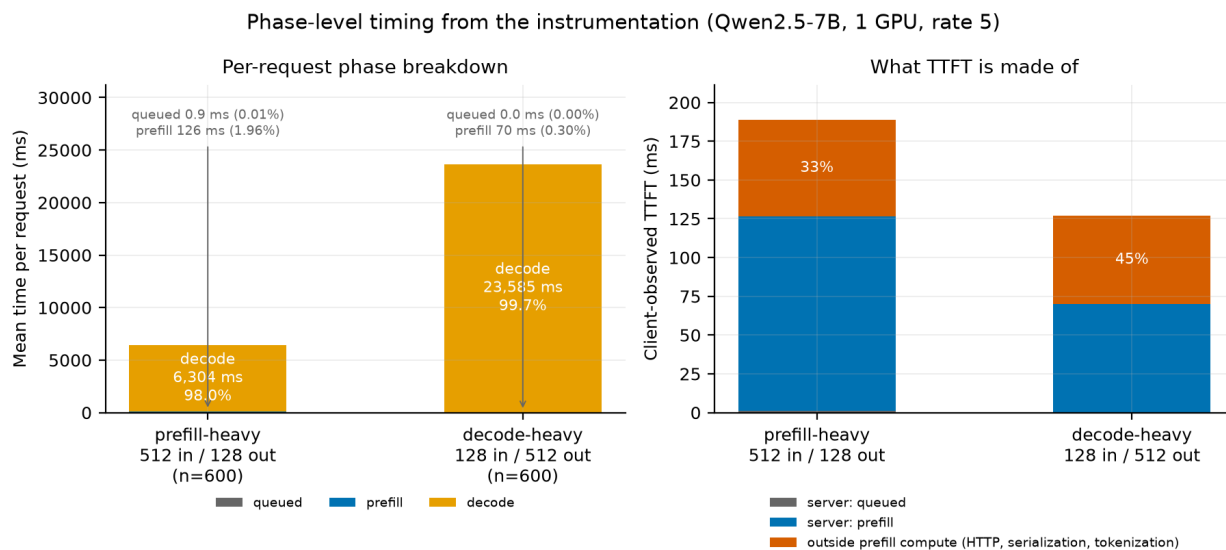


Figure 6.

4.3 Effect of dataset and model size

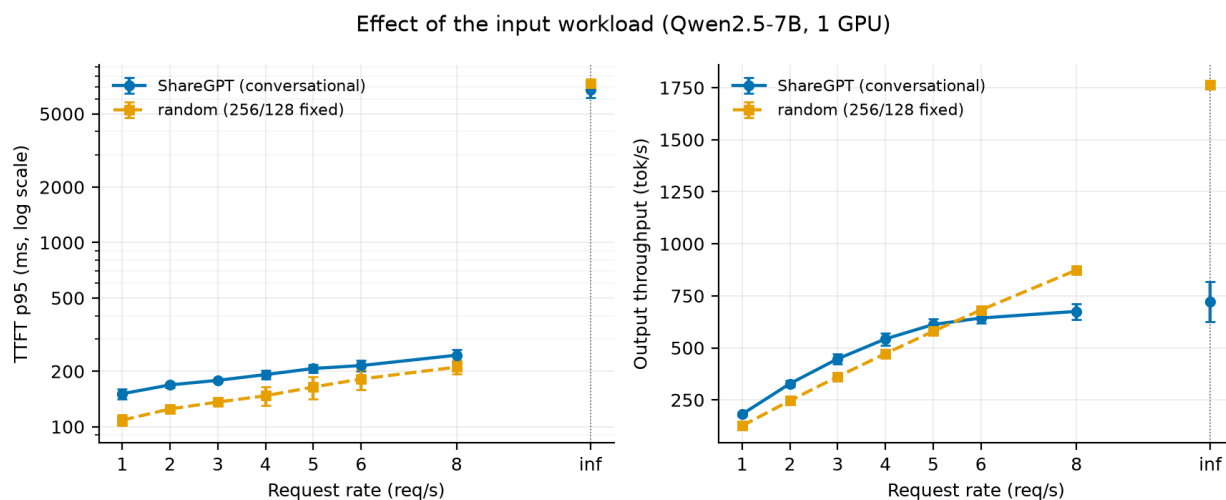


Figure 7.

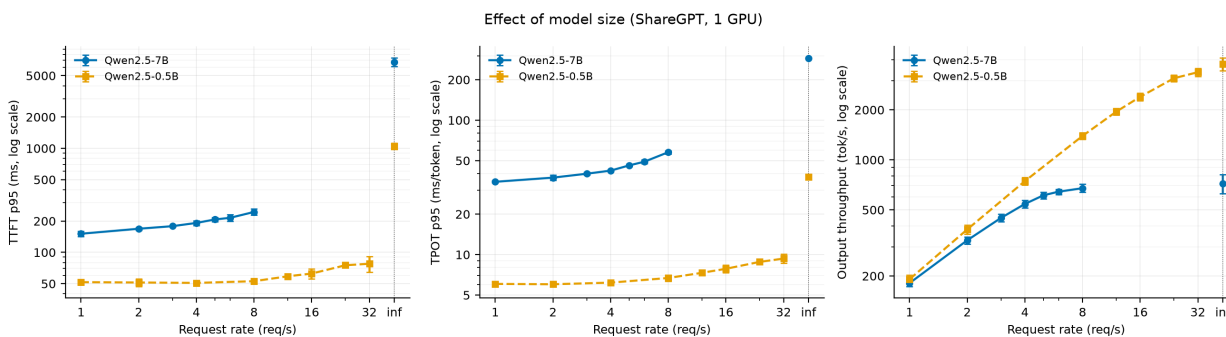


Figure 8.

4.4 Effect of GPU count and parallelism strategy

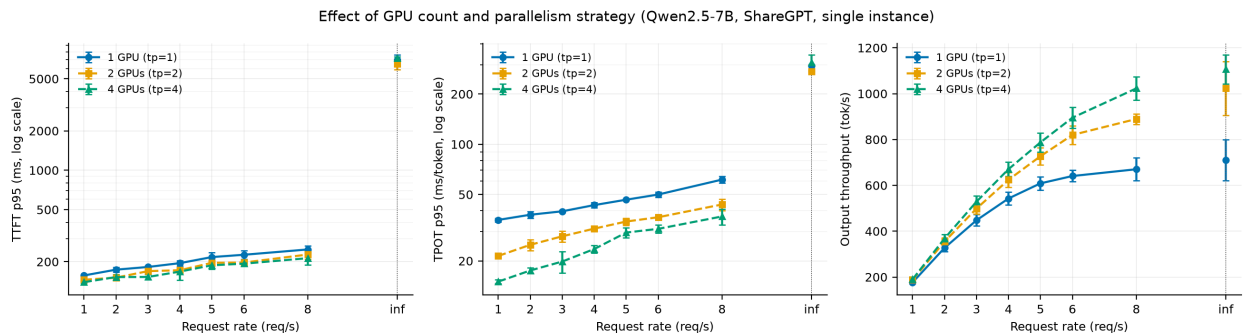


Figure 9.

5. Bottleneck Analysis

5.1 The KV cache is not the constraint

5.2 The mechanism: shorter steps at constant batch

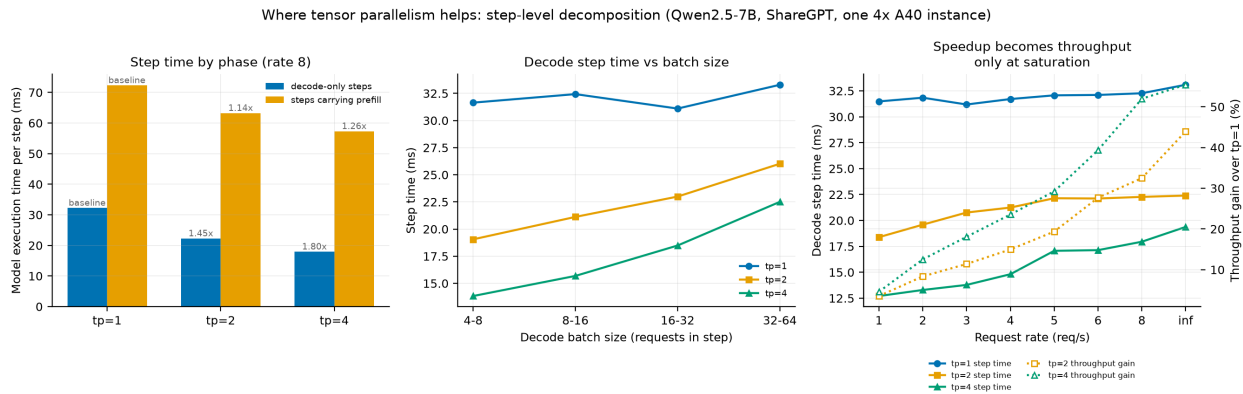


Figure 10.

5.3 Why decode benefits more

5.4 Sublinear scaling and topology

5.5 CPU and framework-bound behaviour

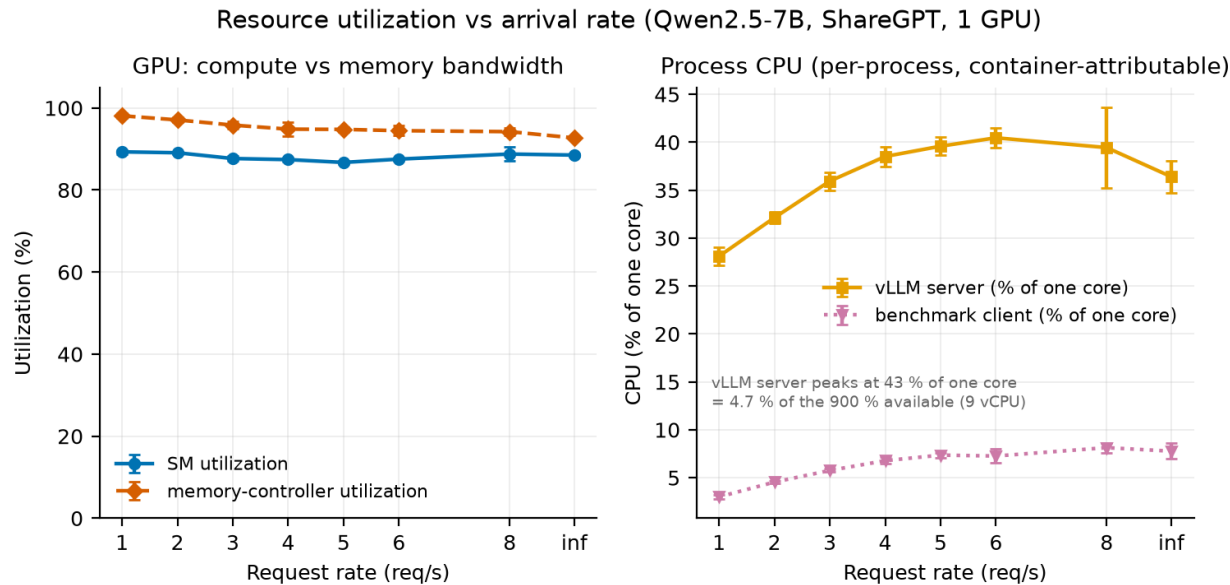


Figure 11.

6. Threats to Validity

7. Conclusion

References

Appendix

- A. Timing-attribution safeguards — `appendix/A_safeguards.md`
- B. Task requirement map — `appendix/B_requirements_map.md`
- C. Environment and reproduction — `appendix/C_environment.md`